Fiche de révision

Architectures parallèles

Multi-cœurs & GPU • Réseaux d'interconnexion

IGL4 / ICE4 – 2024-2025

Comment lire cette fiche

- La fiche suit l'ordre des trois documents du cours. Chaque partie a sa couleur :
  - **Partie 1** : Architectures parallèles (Chapitre 2) ;
  - **Partie 2** : Autres architectures parallèles (multi-cœurs et GPU) ;
  - **Partie 3** : Réseaux d'interconnexion (Chapitre 3).
- Les encadrés ont toujours le même rôle :
  - **Définition** : la définition à savoir par cœur ;
  - **Pour comprendre** : l'intuition derrière une notion ou une formule ;
  - **Exemple** : les machines et cas concrets cités en cours ;
  - **Attention** : les pièges fréquents ;
  - **À retenir** : le résumé de fin de partie, à relire avant l'examen.
- La page 19 regroupe toutes les formules des topologies. La dernière page propose des questions de révision avec réponses.

Contents

Partie 1 – Chapitre 2 – Architectures parallèles 3

1 Quatre décennies de calcul 3

2 La classification de Flynn (1966) 4

3 SIMD et MIMD 4

3.1 Architecture SIMD 4

3.2 Architecture MIMD 5

4 Mémoire partagée et mémoire distribuée 5

4.1 Systèmes à mémoire partagée (SMP) 5

4.2 Systèmes à mémoire distribuée . . . 6
4.3 Architecture hybride (mémoire distribuée-partagée) . . . 6

5 Le classement de Leopold (2001) 7

6 Systèmes parallèles et systèmes distribués 7

7 Réseaux d'interconnexion : critères de classement 7

8 Clusters et grilles 8
8.1 Clusters (grappes) . . . 8
8.2 Grilles (Grids) . . . 8

9 Machines célèbres et ères du calcul parallèle 9

Partie 2 – Autres architectures parallèles – Multi-cœurs et GPU 11

10 Les processeurs multi-cœurs 11

11 Les GPU (Graphics Processing Unit) 11
11.1 La mémoire graphique . . . 12
11.2 Carte graphique dédiée et GPU intégré (IGP) . . . 12
11.3 CPU et GPU . . . 12

12 Programmer un GPU : CUDA 12

Partie 3 – Chapitre 3 – Réseaux d'interconnexion 14

13 Que relie le réseau ? 14

14 Réseaux d'une machine à mémoire partagée 15

15 Réseaux d'une machine à mémoire distribuée 15
15.1 Le vocabulaire des graphes . . . 15
15.2 Chaîne linéaire (grille 1D) . . . 16
15.3 Anneau (réseau torique 1D) . . . 16
15.4 Grille 2D ($q_1 \times q_2$) . . . 16
15.5 Grille 2D torique . . . 17
15.6 Grille 3D (maille) . . . 17
15.7 Arbre binaire (h niveaux) . . . 17
15.8 Hypercube de dimension d . . . 17

16 Fiche formules : toutes les topologies 19

Questions de révision 20

Partie 1 – Chapitre 2

Architectures parallèles

1 Quatre décennies de calcul

Les informaticiens distinguent **4 domaines de calcul**, un par décennie.

	Batch	**Time-sharing**	**Desktop**	**Network**
Décennie	1960s	1970s	1980s	1990s/2000s
Emplacement	Salle machine (ordinateur)	Salle de terminaux (mini-ordi)	Bureau (micro-ordi)	Mobile
Utilisateurs	Experts	Spécialistes	Individus	Groupes
Connectivité	Aucune	Câble périphérique	LAN	Internet / Wireless
Propriétaires	Centre de calcul	Divisional IS shops	Utilisateurs	N'importe qui

Le calcul parallèle en bref

- Le domaine est en **évolution continue**. Il s'est fortement développé **depuis 1985**, à cause de la demande croissante en puissance de calcul.
- Historiquement, il est né avec la construction des **systèmes d'exploitation**. Les machines à plusieurs processeurs sont apparues ensuite.
- On classe les machines parallèles en **deux grandes catégories** :
  - les machines à **mémoire partagée**, qui incluent les machines vectorielles ;
  - les machines à **mémoire distribuée**.
- Il existe aussi des **machines synchrones** : un très grand nombre (jusqu'à **65 536**) d'éléments de calcul de faible puissance.
- Le nombre de processeurs varie de **quelques dizaines** (mémoire partagée) à **des centaines de milliers** (MPP, système massivement parallèle).
- Aujourd'hui, ce sont les **clusters** (grappes) qui dominent.

2 La classification de Flynn (1966)

C'est la classification **la plus répandue**, mais elle est **simpliste**. Elle compte deux flux, et chacun peut être *simple* ou *multiple*.

Définition : les deux flux

- **Flux d'instructions** : la séquence d'instructions exécutées par la machine.
- **Flux de données** : la séquence des données appelées par le flux d'instructions.

	1 flux de données (SD)	**Plusieurs flux de données (MD)**
1 flux d'instructions (SI)	**SISD** – machines séquentielles	**SIMD** – machines vectorielles
Plusieurs flux d'instructions (MI)	**MISD** – aucune machine réelle	**MIMD** – multiprocesseurs

- **SPMD** (Single Program Multiple Data) est une **variante de MIMD** : le **même programme** tourne sur des **données différentes**.
- On a l'ordre : **SIMD** < **SPMD** < **MIMD**.

Attention

MISD existe dans la classification, mais **aucune machine réelle** ne l'a concrétisé. La classification de Flynn reste **insuffisante** pour classer finement les architectures parallèles.

3 SIMD et MIMD

3.1 Architecture SIMD

Définition : SIMD

Un **frontal** et un ensemble de **processeurs identiques synchronisés**. Tous les processeurs exécutent **la même opération** au même moment, mais **sur des données différentes**.

- Chaque processeur a une petite **mémoire locale**, et les processeurs sont reliés par un **réseau d'interconnexion**.
- L'architecture convient aux **calculs numériques réguliers**, comme le **traitement d'images**.
- Elle est **en voie de disparition**. Elle demande des processeurs **spécifiques**, pas ceux du marché. Elle est donc dépassée par l'évolution des processeurs du marché et par les machines MIMD qui les utilisent.

Exemple s : SIMD

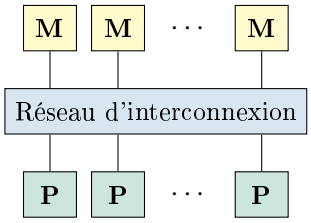
Cray-1 (1976), NEC SX-5, Tera/Cray SV1, Connection Machine CM-1 (1985).

3.2 Architecture MIMD

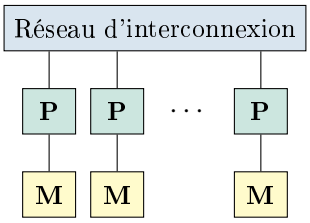
Définition : MIMD

C'est la classe la **plus répandue**. Chaque processeur exécute **son propre code** sur **ses propres données**, de façon **asynchrone et indépendante**.

Il y a deux sous-classes. La différence tient à la place de la mémoire :



MIMD à mémoire partagée
(tous les P accèdent aux mêmes M)



MIMD à mémoire distribuée
(chaque P a sa propre M)

SIMD et MIMD côte à côte

	SIMD	MIMD
Instructions	Une seule, la même pour tous	Chaque processeur a son code
Exécution	Synchronisée	Asynchrone, indépendante
Processeurs	Spécifiques	Ceux du marché
Usage	Calcul régulier (images)	Usage général
Avenir	En voie de disparition	Classe la plus répandue

4 Mémoire partagée et mémoire distribuée

4.1 Systèmes à mémoire partagée (SMP)

Définition : SMP (Symmetric Multiprocessors)

Les processeurs se coordonnent à travers une **mémoire globale partagée** par tous. Ces systèmes comptent en général **entre 2 et 64 processeurs**.

- **Avantage** : la programmation est **relativement facile**, car il n'y a **pas de communications explicites** à écrire.
- **Inconvénient : l'extensibilité**. Le coût est très élevé et la réalisation complexe : une mémoire partagée par plusieurs dizaines de processeurs est très difficile à gérer. Les concepteurs se sont donc tournés vers le « distribué ».

Exemple s : SMP

PC bi-processeurs, Cray SV1, SGI Origin3000, un nœud de l'IBM SP3.

Organisation de la mémoire : UMA ou NUMA

	UMA (Uniform Memory Access)	**NUMA** (Non UMA), aussi appelé **cc-NUMA**
Temps d'accès	**Le même** pour tous les processeurs et toutes les cases mémoire	**Dépend de la distance** entre le processeur et le module mémoire
Construction	Une seule mémoire commune	**Plusieurs SMP reliés**. Chaque SMP peut accéder à la mémoire des autres
Extensibilité	Limitée	**Extensible**
Autre nom / exemple	Aussi nommé **SMP**	SGI Origin3000

4.2 Systèmes à mémoire distribuée

- La notion de **mémoire globale n'existe pas**.
- Ce sont les **seules** machines capables de fournir de **très hautes performances**, grâce à leur **extensibilité**.
- Exemples des années 90 : systèmes à base de **Transputers**, Tera/Cray **T3E**, IBM **SP3**.

Définition : CLUMPS (Clusters of SMP)

Ce sont des machines MIMD construites à partir de SMP. La mémoire est **partagée à l'intérieur de chaque SMP** et **distribuée entre les SMP**.
Exemple : l'IBM SP3, avec des nœuds SMP de 16 processeurs reliés par un réseau dédié.

Attention : p processeurs ne donnent pas un programme p fois plus rapide

Les processeurs doivent **échanger des données**, et les **coûts de communication sont très importants**. Ces échanges se font au détriment de la performance globale.
⇒ **Optimiser les mouvements de données** est la clé de la performance sur les machines à mémoire distribuée.

4.3 Architecture hybride (mémoire distribuée-partagée)

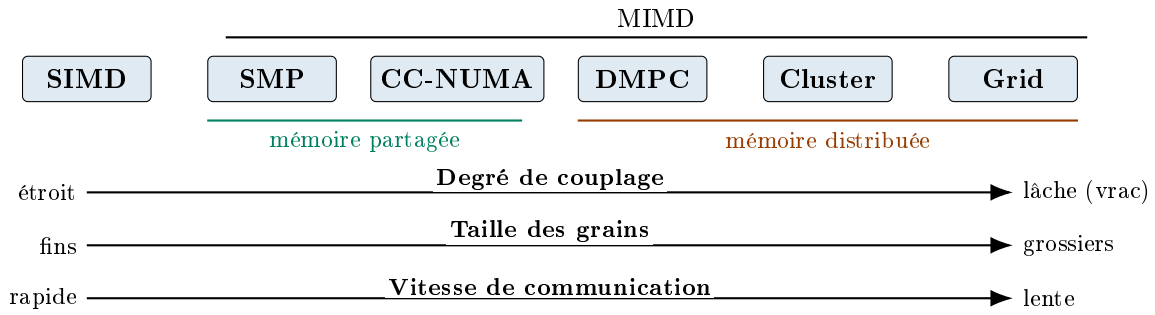
- En réalité, beaucoup de machines couvrent **plus d'une catégorie**. Elles combinent les avantages des deux modèles.
- La mémoire est **physiquement distribuée**. Le **matériel** peut suivre le modèle **passage de messages**, et la **programmation** suit le modèle **mémoire partagée**.
- Exemple : **SGI Origin2000**.

Les deux modèles côte à côte

	Mémoire partagée (SMP)	**Mémoire distribuée**
Mémoire globale	Oui	Non : chaque processeur a la sienne
Communication	Implicite, via la mémoire	Explicite, par messages sur le réseau
Programmation	Facile	Plus difficile
Extensibilité	Faible (2 à 64 processeurs)	Forte, très hautes performances
Exemples	PC bi-pro, Cray SV1, SGI Origin3000	Transputers, Cray T3E, IBM SP3

5 Le classement de Leopold (2001)

Ce classement range les architectures sur un axe. Retenez le **sens des trois flèches**.



Pour comprendre

De gauche à droite, les machines sont de plus en plus « éloignées » les unes des autres, de la puce jusqu'à Internet. Le couplage devient lâche, la communication devient lente, et on ne peut donc paralléliser que de **gros morceaux** de travail (grain grossier).

6 Systèmes parallèles et systèmes distribués

Système parallèle	**Système distribué**
Ensemble d'éléments de calcul qui peuvent **communiquer et coopérer** pour **résoudre rapidement de grands problèmes**.	Ensemble de **processeurs autonomes**, **sans mémoire primaire partagée**, qui coopèrent par **envoi de messages** à travers un réseau.

	Pas distribué	**Distribué**
Pas parallèle	PC / stations qui ne communiquent pas	Stations à **longue distance** (communications trop lentes)
Parallèle	Machines vectorielles, SMP	Stations sur réseau local ou spécialisé, **CLUMPS**

7 Réseaux d'interconnexion : critères de classement

Critère	Possibilités
Mode d'opération	synchrone / asynchrone
Stratégie de contrôle	centralisée / décentralisée
Technique de commutation (switching)	circuit / paquet
Topologie	statique / dynamique

- Un réseau se représente par un **graphe** : les **sommets** sont les processeurs et les **arêtes** les liens de communication.
- La plupart des architectures utilisent des topologies **régulières et statiques**.
- Un réseau **dynamique** établit les connexions **à la volée**, quand les messages arrivent sur les liens.
- Topologies **statiques** courantes : anneau, grille 2D, hypercube, arbre, graphe complet.
- Topologies **dynamiques** : crossbar, multi-étages. *Elles sont détaillées dans la Partie 3.*

8 Clusters et grilles

8.1 Clusters (grappes)

Pourquoi ? Les supercalculateurs spécialisés (Cray/SGI, T3E) étaient trop chers et réservés à une minorité. Les progrès des réseaux et les stations de travail bon marché ont fait **naître les clusters dans les années 90**.

Définition : cluster

Une collection de **nœuds faiblement couplés**, connectés par un **réseau d'interconnexion**. Un nœud peut être un PC, une station de travail ou même un système multiprocesseurs.

- **Homogène** : tous les nœuds ont la même architecture et le même système d'exploitation. Sinon, le cluster est **hétérogène**.
- Le réseau peut être un **LAN rapide**. Il faut une **bande passante élevée** et une **latence faible**.
- C'est un moyen **économique** d'obtenir de la haute performance.
- Les clusters dédiés utilisent souvent **Fast Ethernet** ou **Myrinet**.

Exemple s : clusters (2006)

	Tera-10	**TSUBAME Grid Cluster**
Statut	Plus grand d'Europe, n° 5 du Top500	Plus grand du Japon, n° 7 du Top500
Performance	42,9 TFlops	38,18 TFlops
Processeurs	8704 Itanium 2 à 1,6 GHz	10 368 Opteron à 2,4 GHz
Réseau	Quadrics	Infiniband
Constructeur, lieu	Bull, France (CEA)	NEC, Japon (TIT)

8.2 Grilles (Grids)

L'image à retenir (Ian Foster, 1998) : la grille est comme le **réseau électrique**. On branche la prise et on reçoit la puissance, sans savoir quelle centrale la produit.

Définition : grille de calcul

Une collection de **clusters hétérogènes**, **dispersés géographiquement** et connectés par des réseaux **WAN ou Internet**.

- C'est une infrastructure qui fournit calcul, stockage et communication de façon **transparente**, quand et où l'utilisateur les demande.
- Le **middleware** masque l'architecture. L'utilisateur soumet seulement des requêtes, sans savoir où sont ses données ni où tournent ses calculs.
- **Caractéristiques** : forte hétérogénéité, structure hiérarchique, distribution géographique des ressources, décentralisation.

Exemple s : grilles

GRID5000 (France) : chaque nœud est un cluster de 256 à 1024 CPU, et le débit peut atteindre 10 GB/s. **TeraGrid** (USA).

Cluster et grille côte à côte

	Cluster	**Grille**
Composé de	Nœuds : PC, stations, multiprocesseurs	**Clusters**
Réseau	LAN rapide (Fast Ethernet, Myrinet)	WAN / Internet
Hétérogénéité	Homogène ou hétérogène	**Forte** hétérogénéité
Géographie	Au même endroit	Dispersée
Apparition	Années 90	Années 2000 (Foster, 1998)

9 Machines célèbres et ères du calcul parallèle

Machine	Perf.	Processeurs	Réseau / mémoire	Constructeur
Cray-2 (1985–1990)	1,2 GFlops	4 vectoriels	Mémoire partagée	Cray
CM-5/1024 MPP (1992–1993)	59,7 GFlops	1024 SPARC	Réseau en arbre plat	TMC
Numerical Wind Tunnel (1995)	170,4 GFlops	140 vectoriels	Switch crossbar	Fujitsu
Earth Simulator MPP (2002–2004)	35,86 TFlops	5120 (640 nœuds × 8 vectoriels)	Switch crossbar (2400 km de câbles)	NEC
BlueGene/L MPP (2005–2006)	280 TFlops	131 072 (2 PowerPC 440 à 700 MHz par nœud)	Réseau **toroïdal 3D** (peer-to-peer)	IBM

	1970s	**1980s**	**1990s**	**2000s**	**Après 2000**
Parallélisme	Processeur	Machine	LAN	WAN	Puce
Architecture	Vectorielle	SMP / MPP	Cluster	Grid	Multi-core
Réseau	Aucun	Bus, switch, grille, hypercube	Ethernet, switch	Internet	Sur la puce
Programmation	Vector, Fortran	C*, C-Linda, Occam	PVM, MPI, HPF	MPI, OpenMP	?

À retenir – Partie 1

- **Flynn** : SISD (séquentiel), SIMD (vectoriel), MISD (aucune machine), MIMD (multiprocesseurs). SPMD est une variante de MIMD. SIMD < SPMD < MIMD.
- **SIMD** : même opération, données différentes, processeurs synchronisés. En voie de disparition.
- **MIMD** : la plus répandue, asynchrone. Elle peut être à mémoire **partagée** ou **distribuée**.
- **SMP** : programmation facile, mais peu extensible (2 à 64 processeurs). **UMA** : même temps d'accès. **NUMA** : le temps dépend de la distance, et l'architecture est extensible.
- **Mémoire distribuée** : seule à offrir de très hautes performances. **Communications = coût principal**.

- **CLUMPS** : partagée dans chaque SMP, distribuée entre les SMP. **Hybride** : mémoire physiquement distribuée, programmée comme partagée.
- **Leopold** : SIMD → SMP → CC-NUMA → DMPC → Cluster → Grid. Le couplage devient lâche, le grain grossier, la communication lente.
- **Cluster** : nœuds faiblement couplés sur un LAN. **Grille** : clusters hétérogènes sur WAN/Internet, transparents grâce au middleware.

Partie 2 – Autres architectures parallèles

Multi-cœurs et GPU

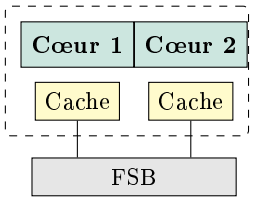
10 Les processeurs multi-cœurs

Définition : multi-cœur

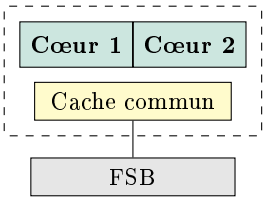
Un processeur composé d'**au moins deux cœurs** (unités de calcul) gravés **sur la même puce**.

Problème	Solution du multi-cœur
On veut plus de puissance, mais monter la **fréquence d'horloge** a ses limites.	**Multiplier la puissance** par le parallélisme, **sans augmenter la fréquence**.
Les systèmes **multiprocesseurs** consomment beaucoup d'énergie et dégagent beaucoup de chaleur.	Les cœurs sont placés **côte à côte sur le silicium**, ce qui **réduit la consommation et la chaleur**.

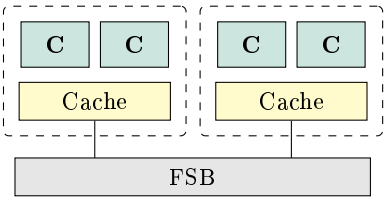
Les trois types d'architectures multi-cœurs



1. **Caches distincts**



2. **Cache commun**



3. **« Quad-Core » Intel**
= 2 puces double-cœur

Attention : le multi-cœur ne suffit pas tout seul

- Les programmes doivent être **repensés**. Sinon, ils n'utilisent **qu'un seul cœur**.
- Très peu de logiciels gèrent le bi-cœur, et ils ne sont pas conçus pour en tirer le maximum.
- Le gain est net pour les applications **gourmandes**, comme les jeux vidéo.
- Les **quadri-cœurs** (4 cœurs) remplacent les bi-cœurs, car le passage ne demande **quasiment aucune modification logicielle**.

11 Les GPU (Graphics Processing Unit)

Définition : GPU (ou VPU, Visual Processing Unit)

Un circuit intégré présent sur une **carte graphique**, qui réalise les **calculs de l'affichage**. Sa structure est **hautement parallèle**.

- Il est efficace pour le **rendu 3D**, la gestion de la **mémoire vidéo** et le **traitement du signal vidéo**.

- Il **libère le CPU** des calculs d'affichage et de la coordination 3D, ce qui **augmente la puissance apparente** de l'ordinateur.
- Il a souvent son propre **radiateur ou ventilateur**.
- La **carte graphique** (adaptateur graphique) est une carte d'extension qui produit l'image affichable. Elle envoie à l'écran les images stockées dans sa mémoire.
- Peu d'entreprises conçoivent des GPU : **NVIDIA, ATI/AMD, Intel, S3 Graphics**.
- **AMD** (ex-ATI) et **NVIDIA** permettent d'utiliser **plusieurs cartes graphiques** dans un même ordinateur, ce qui augmente fortement la puissance.

11.1 La mémoire graphique

- Elle est **presque aussi importante que le GPU**. Elle stocke les données **géométriques et colorimétriques** et les met à disposition du GPU.
- Elle joue un rôle proche de celui de la **RAM pour le CPU**. Sa **vitesse** et sa **taille** influent fortement sur les performances.

	Mémoire partagée	**Mémoire dédiée**
Principe	Une partie de la **RAM centrale** est réservée au GPU	Mémoire **embarquée sur la carte graphique**, ou près du GPU sur un portable
Accès	Via la mémoire de l'ordinateur	**Bus dédié** vers le GPU
Usage	Configurations économiques, bureautique	Performance

11.2 Carte graphique dédiée et GPU intégré (IGP)

	Carte graphique dédiée	**GPU intégré (IGP)**
Emplacement	Carte séparée, reliée à la carte mère	Dans le **northbridge** ou dans le **CPU**
Mémoire	Sa propre mémoire vive	La RAM de l'ordinateur (rarement un peu de mémoire dédiée)
Performance	**La plus puissante**	Moins performante
Points forts	Puissance	Moins chère, plus facile à intégrer, consomme moins
Usage typique	Jeux modernes, calcul	Portables anciens ou bas de gamme, usage sans jeux modernes

Les cartes mères actuelles ont souvent un IGP **et** un ou plusieurs ports pour ajouter une carte dédiée.

11.3 CPU et GPU

	CPU	**GPU**
Transistors	Beaucoup pour les caches et le contrôle	**Beaucoup plus d'ALU**, moins de caches
Bande passante mémoire	Plus faible	**Plus élevée**
Rôle	Les **parties séquentielles**	Les **calculs intensifs**, qu'il accélère

12 Programmer un GPU : CUDA

Le problème : les langages graphiques (**OpenGL, Cg**) obligent à écrire une application scientifique *comme si c'était une application graphique*. Les performances du GPU restent donc difficiles d'accès

pour les applications non graphiques.

La solution : NVIDIA propose **CUDA** (*Compute Unified Device Architecture*). C'est un langage et un logiciel de haut niveau qui exploitent le GPU par la **programmation parallèle avec des threads**.

Définition : CUDA

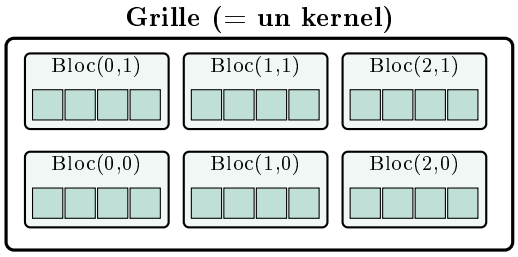
Une **API**, considérée comme une **extension du langage C**. Le logiciel CUDA a **trois modules** :

1. une **interface de programmation** (API) ;
2. un **runtime**, intermédiaire entre l'utilisateur et le driver ;
3. une **bibliothèque** d'outils mathématiques.

Un programme CUDA = code CPU + code GPU

Partie	Où s'exécute-t-elle ?	Comment ?
Serial	CPU (l'*hôte*, host)	Code C classique
Kernel	GPU (le *device*)	Code C exécuté par des **threads parallèles**

Modèle d'exécution : grille, blocs, threads



□ = un **thread**

Les threads d'un même **bloc** :
- **coopèrent** ;
- se **synchronisent** ;
- partagent la **mémoire partagée** (Shared Memory).

1. CUDA exécute une **séquence de kernels** sur le GPU.
2. Chaque **kernel** s'exécute comme une **grille de blocs de threads**.
3. Un **bloc** est une liste de threads qui peuvent **coopérer** : synchronisation et partage de données en **mémoire partagée**.
4. Les blocs s'exécutent comme des **CTA** (*Concurrent Thread Array*) sur les cœurs **SM**.

À retenir – Partie 2

- **Multi-cœur** : au moins 2 cœurs sur la même puce. On gagne en puissance **sans augmenter la fréquence**, avec moins d'énergie et moins de chaleur. Il existe 3 types : caches distincts, cache commun, « Quad-Core » Intel (2 puces double-cœur).
- Il faut **repenser les programmes**, sinon un seul cœur travaille.
- **GPU** : très parallèle, il libère le CPU. Il a plus d'ALU, moins de caches et une plus grande bande passante. Le CPU fait le séquentiel, le GPU le calcul intensif.
- **Mémoire graphique** : partagée (une part de la RAM) ou dédiée (sur la carte, avec un bus dédié). **Carte dédiée** : la plus puissante. **IGP** : moins chère et plus économe.
- **CUDA** (NVIDIA) : une extension du C, avec API, runtime et bibliothèque. Un **kernel** (GPU) forme une **grille de blocs de threads**. Les blocs s'exécutent comme des **CTA** sur les cœurs SM.

Partie 3 – Chapitre 3

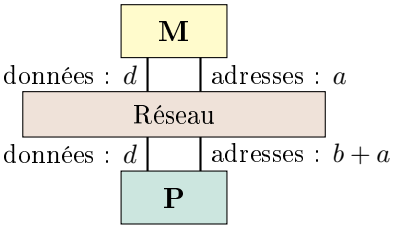
Réseaux d'interconnexion

13 Que relie le réseau ?

Mémoire partagée (SM)	Mémoire distribuée (DM)
Le réseau relie les **bus de données** et les **bus d'adresses** des processeurs à **n'importe quel module mémoire**.	Le réseau **achemine les messages** entre les processeurs : c'est une machine à **passage de messages**.

Mémoire partagée : les bus

- Tous les bus de données ont la **même largeur** d.
- La largeur des bus d'adresses dépend de la **taille du module mémoire**.
- Une adresse est un couple (b, a) : b est l'adresse du **module (banc) mémoire** et a l'adresse de la **case dans ce banc**.



Pour comprendre

Côté processeur, l'adresse doit contenir b (*quel banc ?*) et a (*quelle case ?*), d'où une largeur de $b + a$. Le réseau utilise b pour choisir le banc. Ce banc ne reçoit donc que a.

Mémoire distribuée : les messages

Entête : destinataire, expéditeur, informations	**Corps**

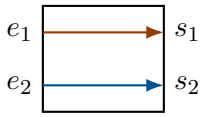
14 Réseaux d'une machine à mémoire partagée

Il y en a trois types, du plus simple au plus élaboré :

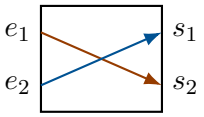
	Zéro étage (bus)	**Un étage (crossbar)**	**Multi-étages**
Mémoire	**Un seul** banc	**Plusieurs** bancs	Plusieurs bancs
Connexion	Tous sur un bus	Chaque P relié à chaque banc : **graphe complet**	Étages de **boîtes d'échange**
Parallélisme	Aucun : conflit sur la mémoire	Accès en parallèle à des bancs **différents**	Accès simultanés possibles s'il n'y a pas de conflit dans une boîte
Conflit	Résolu par **exclusion mutuelle** sur le banc	Exclusion mutuelle sur le **même** banc	**Arbitrage** dans la boîte

Les boîtes d'échange (exchange box)

Une boîte relie **2 entrées** (e_1, e_2) à **2 sorties** (s_1, s_2). Elle a deux positions :



Straight (tout droit)
$e_1 \to s_1$, $e_2 \to s_2$

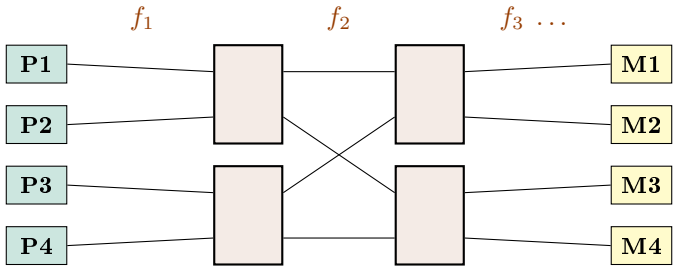


Exchange (croisé)
$e_1 \to s_2$, $e_2 \to s_1$

À retenir : la règle des conflits

Si deux processeurs passent par la **même boîte** et demandent tous les deux la **même position** (straight ou exchange), il n'y a **pas de conflit** : l'accès est simultané. Sinon, il faut un **arbitrage**.

Schéma général d'un réseau multi-étages



Les f_i représentent le **câblage (permutation)** entre deux étages de boîtes.

15 Réseaux d'une machine à mémoire distribuée

15.1 Le vocabulaire des graphes

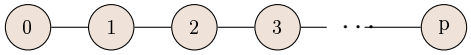
Un réseau est un graphe $G(X, U)$: X est l'ensemble des nœuds (processeurs) et U l'ensemble des arêtes (liens).

Symbole	Nom	Signification
$n = \|X\|$	**Ordre**	**Nombre de processeurs**
$m = \|U\|$	**Nombre d'arêtes**	**Nombre de liens**
Δ	**Degré maximal**	Le plus grand nombre de liens qui touchent un même nœud
D	**Diamètre**	La **plus grande distance** entre deux nœuds. La distance est la longueur de la plus courte chaîne entre eux
K	**Connectivité**	Le **nombre de chemins différents** entre deux nœuds

Pour comprendre : ce que chaque grandeur mesure

- Un petit D veut dire que les messages arrivent vite, en peu de sauts.
- Un petit m et un petit Δ veulent dire un réseau moins cher, avec moins de câbles et moins de ports par nœud.
- Avec $K \geq 2$, les **communications en parallèle** deviennent possibles, et le réseau résiste mieux aux pannes. Avec $K = 1$, il n'y a **pas de parallélisme possible**.

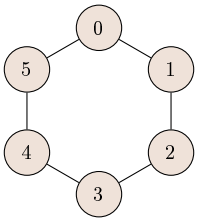
15.2 Chaîne linéaire (grille 1D)



- $n = p \quad m = p - 1 \quad D = p - 1$
- $\Delta = 2$: un nœud d'extrémité a le degré 1, un nœud intérieur le degré 2.
- $K = 1$, donc **pas de parallélisme** possible.

Intuition : pour aller d'un bout à l'autre, on traverse tous les liens, et il n'y a qu'un seul chemin.

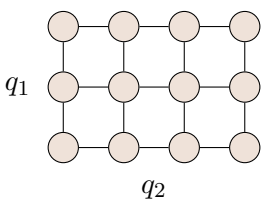
15.3 Anneau (réseau torique 1D)

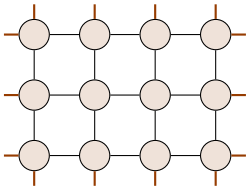


- $n = p \quad m = p \quad D = E(p/2) \quad \Delta = 2$
- $K = 2$, donc le **parallélisme est possible**.

Intuition : c'est la chaîne plus un lien qui la referme. On peut tourner dans les deux sens, d'où 2 chemins, et on n'a jamais à parcourir plus de la moitié de l'anneau. E désigne la partie entière.

15.4 Grille 2D ($q_1 \times q_2$)





grille 2D **torique**
chaque lien coloré rejoint le bord opposé

- $n = q_1 q_2$
- $m = q_1(q_2 - 1) + q_2(q_1 - 1) = 2q_1q_2 - (q_1 + q_2)$
- $D = q_1 + q_2 - 2$
- $\Delta = 4$: un coin a le degré 2, un bord le degré 3, un nœud intérieur le degré 4.
- $K \geq 2$, donc le parallélisme est possible.
- Chaque nœud est numéroté par un **couple** de coordonnées.

Intuition : chacune des q_1 lignes a $q_2 - 1$ liens horizontaux, et chacune des q_2 colonnes a $q_1 - 1$ liens verticaux. Le chemin le plus long va d'un coin au coin opposé : $(q_1 - 1) + (q_2 - 1)$ sauts.

15.5 Grille 2D torique

On ajoute des liens de bouclage (en couleur sur le schéma) sur chaque ligne et chaque colonne.

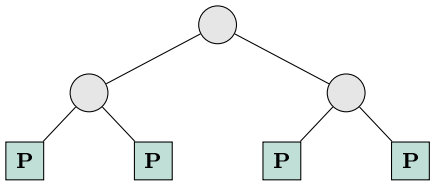
- $n = q_1 q_2 \quad m = 2q_1q_2 \quad D = E(q_1/2) + E(q_2/2) \quad \Delta = 4 \quad K \geq 2$

Intuition : chaque nœud a maintenant 4 voisins, donc $m = 4n/2 = 2q_1q_2$. Dans chaque direction, on ne fait jamais plus de la moitié du tour, comme pour l'anneau.

15.6 Grille 3D (maille)

- $K \geq 3$.
- Chaque nœud est numéroté par un **triplet** de coordonnées.

15.7 Arbre binaire (h niveaux)



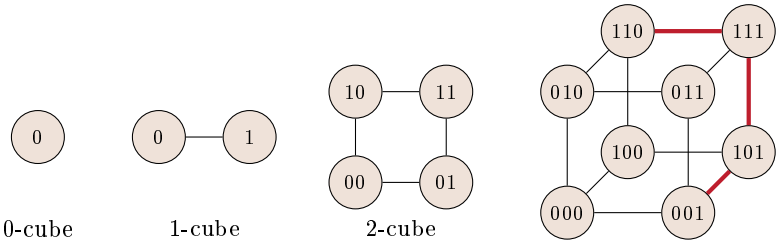
○ = élément de commutation
□ = processeur (feuilles)

- $n = 2^h - 1$ nœuds $\quad m = 2^h - 2$
- $\Delta = 3 \quad D = 2(h - 1)$
- $K = 1$

Intuition : un nœud interne a 1 parent et 2 enfants, d'où $\Delta = 3$. Le plus long trajet va d'une feuille à une autre en passant par la racine : $h - 1$ sauts pour monter, $h - 1$ pour descendre. Dans un arbre, le chemin est unique, d'où $K = 1$.

15.8 Hypercube de dimension d

Construction par récurrence : un 0-cube est **un seul processeur**. Un d-cube s'obtient en **superposant deux** $(d-1)$**-cubes** et en reliant chaque sommet à son correspondant dans l'autre cube.



0-cube 1-cube 2-cube

3-cube (en rouge : un chemin 001 → 110)

Grandeur	Récurrence	Résultat
Nœuds	$n_d = 2\,n_{d-1}$	$n = 2^d$
Liens	$m_d = 2\,m_{d-1} + n_{d-1}$	$m = d \cdot 2^{d-1}$
Degré	$\Delta_d = \Delta_{d-1} + 1$	$\Delta = d$ (le degré de chaque nœud)
Diamètre	$D_d = D_{d-1} + 1$	$D = d$

Pour comprendre : d'où viennent les récurrences ?

On copie le $(d-1)$-cube : le nombre de nœuds et de liens double. On ajoute ensuite **un lien par nœud de la copie**, soit n_{d-1} liens de plus. Chaque nœud gagne un voisin, donc Δ augmente de 1. Le chemin le plus long gagne un saut, donc D augmente de 1.
Vérification : d liens par nœud, 2^d nœuds, et chaque lien est compté deux fois : $m = d \cdot 2^d/2 = d \cdot 2^{d-1}$.

Numérotation (code de Hamming)

- Chaque nœud reçoit l'écriture **binaire sur d bits** d'un nombre de 0 à $2^d - 1$.
- Deux nœuds sont **voisins** si leurs numéros diffèrent d'**un seul bit**.
- La **distance** entre deux nœuds est le **nombre de bits différents** (distance de Hamming).
- Comme $D = d$, il existe **plusieurs chemins parallèles** entre deux nœuds.

Exemple : combien de chemins parallèles entre 001 et 110 dans le 3-cube ?

001 et 110 diffèrent sur **3 bits**, donc la distance vaut 3. Pour chaque chemin, on change un bit à chaque saut, et on choisit le premier bit à modifier :

On change d'abord le bit de gauche : $001 \to 101 \to 111 \to 110$
On change d'abord le bit du milieu : $001 \to 011 \to 010 \to 110$
On change d'abord le bit de droite : $001 \to 000 \to 100 \to 110$

On obtient **3 chemins parallèles**, qui ne partagent aucun nœud intermédiaire.

16 Fiche formules : toutes les topologies

Topologie	n (nœuds)	m (liens)	D (diamètre)	Δ (degré)	K (connect.)
Chaîne linéaire	p	$p-1$	$p-1$	2	1
Anneau	p	p	$E(p/2)$	2	2
Grille 2D	$q_1 q_2$	$2q_1q_2-(q_1+q_2)$	q_1+q_2-2	4	≥ 2
Grille 2D torique	$q_1 q_2$	$2q_1q_2$	$E(q_1/2)+E(q_2/2)$	4	≥ 2
Grille 3D	–	–	–	–	≥ 3
Arbre binaire	2^h-1	2^h-2	$2(h-1)$	3	1
Hypercube	2^d	$d\,2^{d-1}$	d	d	d (*)

(*) Le cours ne donne pas cette valeur directement. On la déduit de l'exemple du 3-cube, qui a 3 chemins parallèles, soit autant que la dimension.
Pour la grille 3D, le cours ne donne que $K \geq 3$.

À retenir – Partie 3

- En **SM**, le réseau relie les bus aux bancs mémoire. Une adresse est un couple (b, a), et la largeur des bus d'adresses vaut $b+a$ côté processeur et a côté mémoire. En **DM**, le réseau achemine des **messages** : entête (destinataire, expéditeur, informations) puis corps.
- Les réseaux SM vont du plus simple au plus élaboré : **bus** (1 banc, exclusion mutuelle), **crossbar** (graphe complet, parallèle sur des bancs différents), **multi-étages** (boîtes d'échange straight ou exchange, arbitrage en cas de conflit).
- Pour les réseaux DM, il faut savoir les **5 grandeurs** n, m, D, Δ, K de chaque topologie (voir la fiche formules). Avec $K=1$, pas de parallélisme : c'est le cas de la chaîne et de l'arbre.
- **Hypercube** : 2^d nœuds, $d\,2^{d-1}$ liens, $D=\Delta=d$. Les voisins diffèrent d'un bit. La distance est la distance de Hamming. Il y a plusieurs chemins parallèles.

Questions de révision (avec réponses)

Q1. Quelles sont les 4 classes de Flynn, et laquelle n'a aucune machine réelle ?
SISD, SIMD, MISD, MIMD. C'est **MISD** qui n'a aucune machine réelle.

Q2. Pourquoi le SIMD est-il en voie de disparition ?
Il demande des processeurs **spécifiques**, dépassés par les processeurs du marché et par les machines MIMD qui les utilisent.

Q3. Quelle est la différence entre UMA et NUMA ?
En UMA, le temps d'accès est le même partout. En NUMA, il dépend de la distance entre le processeur et le module mémoire. NUMA relie plusieurs SMP et il est extensible.

Q4. Pourquoi p processeurs ne rendent-ils pas un programme p fois plus rapide ?
Il faut échanger des données, et le **coût des communications** est très élevé.

Q5. Qu'est-ce qu'un CLUMPS ? Donnez un exemple.
Un cluster de SMP : la mémoire est partagée dans chaque SMP et distribuée entre les SMP. Exemple : l'IBM SP3, avec des nœuds de 16 processeurs.

Q6. Quelle différence entre un cluster et une grille ?
Un cluster est fait de nœuds faiblement couplés sur un LAN rapide. Une grille est faite de clusters hétérogènes dispersés, reliés par un WAN ou Internet, et rendus transparents par le middleware.

Q7. Pourquoi le multi-cœur plutôt que le multiprocesseur ou la hausse de fréquence ?
Il donne plus de puissance **sans augmenter la fréquence**, avec moins de consommation et moins de chaleur.

Q8. Qu'est-ce qu'un kernel CUDA, et comment s'exécute-t-il ?
C'est du code C exécuté sur le GPU, sous forme d'une **grille de blocs de threads**. Les threads d'un bloc se synchronisent et partagent la mémoire partagée. Les blocs s'exécutent comme des CTA sur les cœurs SM.

Q9. Dans un réseau multi-étages, quand y a-t-il conflit dans une boîte d'échange ?
Quand deux processeurs qui passent par la même boîte demandent des positions **différentes** (l'un straight, l'autre exchange). Il faut alors un arbitrage.

Q10. Pour une grille 2D 4×5, calculez n, m, D.
$n = 20$; $m = 2 \cdot 20 - (4 + 5) = 31$; $D = 4 + 5 - 2 = 7$.

Q11. Pour un hypercube de dimension 4, calculez n, m, D, Δ.
$n = 2^4 = 16$; $m = 4 \cdot 2^3 = 32$; $D = 4$; $\Delta = 4$.

Q12. Quelle est la distance entre 0110 et 1011 dans un 4-cube ?
Les bits diffèrent en positions 1, 2 et 4, donc la distance vaut **3**.